Skill-Knowledge-Augmented Agents on AssetOpsBench

Vera Mazeeva
Department of Computer Science
Columbia University
New York, NY, USA
vmm2146@columbia.edu

Mana Abbaszadeh
Department of Computer Science
Columbia University
New York, NY, USA
ma4845@columbia.edu

Shrey Arora
Department of Computer Science
Columbia University
New York, NY, USA
ska2153@columbia.edu

Sanskruti Shejwal
Department of Computer Science
Columbia University
New York, NY, USA
ss7561@columbia.edu

***Abstract*—Industrial operations and maintenance (O&M) agents must diagnose equipment faults and recommend maintenance actions from heterogeneous inputs such as sensor streams, failure-mode libraries, operating ranges, and work-order histories. Existing AssetOpsBench baselines execute static tool pipelines: they often invoke Deep-TSFM statistical time-series validation even when a cheaper failure-mode sensor reasoning (FMSR) step has already produced a high-confidence diagnosis. This report presents a skill-augmented and knowledge-aware agent that replaces free-form tool use with deterministic skill execution, scoped knowledge plugins, and a confidence gate between first-pass FMSR diagnosis and expensive Deep-TSFM validation. The optimization is inference-only: no model parameters are trained or fine-tuned. On the final evaluated run, built from a 54-scenario stratified input slice with 46 successfully graded scenarios, the best gated condition at $\theta = 0.8$ reaches a 30.4% overall-correct rate, compared with 13.0% for the static-tool baseline and 0.0% for raw LLM prompting. The same setting improves agent-sequence correctness from 6.5% to 88.9% and reduces hallucination rate from 93.5% to 35.6%, showing that structured orchestration and confidence-based skipping can improve both reliability and efficiency in industrial multi-agent workflows.**

***Index Terms*—LLM agents, industrial operations and maintenance, AssetOpsBench, skill-based agents, knowledge plugins, time-series forecasting, confidence gating, inference optimization**

I. Introduction

A. Background and Motivation

Industrial operations and maintenance (O&M) systems must continuously monitor complex physical assets, such as chillers, air-handling units (AHUs), pumps, motors, and bearings, from heterogeneous data sources including sensor streams, failure-mode libraries, and maintenance logs. Translating these signals into timely maintenance decisions is among the most demanding multi-step reasoning tasks in applied AI.

LLM-based agents have emerged as a compelling paradigm for industrial O&M, capable of orchestrating multiple specialized tools within a single reasoning loop [5]. AssetOpsBench [1] formalises this challenge as a benchmark of 141 human-authored scenarios spanning sensor-query mapping, anomaly detection, failure diagnosis, forecasting, and work-order generation, backed by four MCP agent servers, like IoT, TSFM, FMSR, and WO, called via a planner-executor layer.

Despite their promise, current baseline agents operate through *static pipelines* that invoke all components unconditionally, regardless of task complexity or intermediate results. Consider the representative query: *"Why is Chiller 6 behaving abnormally, and do we need a work order?"* The baseline always: (1) retrieves all sensor data via IoT, (2) runs full TSFM statistical anomaly detection across every sensor, (3) invokes FMSR to map anomalies to failure modes, and (4) generates a work order via WO. This produces four concrete inefficiencies: the TSFM agent, which is the most expensive component, requiring ML time-series inference, runs even when FMSR already returns a high-confidence diagnosis; the LLM selects tools step-by-step without explicit workflow structure, yielding inconsistent trajectories; all sensor metadata is fetched upfront regardless of relevance; and reasoning never terminates early, even when sufficient confidence is already reached. These inefficiencies are most severe in fault-diagnosis tasks, which invoke all four agents in sequence, but manifest across all 141 scenarios.

B. Problem Statement

We identify a concrete, measurable inefficiency: the static AssetOpsBench baseline unconditionally invokes Deep-TSFM statistical validation on fault-diagnosis tasks even when the FMSR agent has already produced a high-confidence diagnosis. Our trajectory audit of the plan-execute baseline across 152 scenarios confirms that **35.5% of all trajectories** contain unnecessary TSFM tool calls, 54 violations in total, of which 52 (96%) are wrong-domain calls where ML time-series inference was entirely irrelevant to the correct answer.

Our problem is therefore: *how can an industrial O&M agent dynamically decide when not to invoke an expensive analysis component, based on the confidence of a cheaper*

preceding step, without sacrificing diagnostic accuracy or task completion rate?

C. Contributions and Scope

We make three primary contributions:

1) **TSFM Redundancy Audit.** We instrument the AssetOpsBench plan-execute baseline with a trajectory logging and TSFM misuse auditor and quantify unnecessary invocations across all 152 accessible scenarios, categorising violations as *fault-diagnosis misuse* (FMSA scenarios calling TSFM when domain knowledge suffices) and *wrong-domain calls* (work-order and multi-agent scenarios routing to TSFM because the planner associates anomaly-related keywords with ML inference).
2) **Skill-Augmented Agent Framework.** We introduce a modular agent built on Python-based skill plugins and knowledge plugins. Skills encapsulate deterministic, multi-step workflows (data retrieval, lightweight anomaly detection, root cause analysis, conditional Deep-TSFM validation, and work order generation). Knowledge plugins inject only the domain context each skill requires at runtime, keeping context windows lean and reducing hallucination.
3) **Confidence-Gated TSFM Invocation.** We implement a confidence-gating mechanism between the FMSR root-cause analysis step and the Deep-TSFM validation step. If FMSR confidence $\geq \theta$, Deep-TSFM is skipped and the pipeline proceeds directly to work-order generation. We sweep $\theta \in \{0.50, 0.60, 0.65, 0.70, 0.80, 0.90, 0.95\}$ and report the accuracy-efficiency trade-off across 12 ablation conditions and 54 stratified AssetOpsBench scenarios ($12 \times 54 = 648$ ablation rows per full run; Tables II–III use 46 successfully graded scenarios per condition).

The scope of this work is bounded by the AssetOpsBench benchmark [1] and the four MCP agent servers it provides. We do not train new models; all improvements derive from better orchestration logic. Evaluation uses an LLM judge (WatsonX `llama-3-3-70b-instruct`) grading six qualitative dimensions per trajectory: task completion, data retrieval accuracy, result verification, agent-sequence correctness, clarity and justification, and hallucination rate.

II. Literature Review

A. LLM-Based Reasoning and Tool Use

ReAct [5] introduced interleaved reasoning traces and tool-use actions within a single LLM forward pass, establishing the dominant paradigm for agent-based task completion. However, ReAct lacks explicit workflow structure: the LLM selects tools step-by-step without a pre-defined execution graph, producing inconsistent trajectories that are difficult to audit or optimise. Our work addresses this by replacing free-form tool calling with a deterministic skill executor whose control flow is fully specified by the skill registry.

B. Multi-Agent Orchestration

DeepAgents [2] provides structured orchestration with sub-agent delegation but operates at the level of individual tool calls, offering no abstractions for reusable multi-step workflows. Unconditional invocation inefficiencies therefore persist within each sub-agent loop. Magentic-One [4] adds task and progress ledgers suited to open-ended web tasks, but its coordination overhead is unjustified for AssetOpsBench's well-defined four-agent taxonomy. Our skill abstraction layer pre-specifies both which agents to call and the conditional logic governing whether each skill executes at all.

C. Skill Composition and Modular Agents

SkillsBench [3] shows that skill-based decomposition reduces reasoning complexity and improves generalisation over end-to-end tool-calling agents, but its skill libraries are static: skills execute unconditionally without adaptive orchestration based on intermediate confidence. SkillNet [6] provides large-scale skill infrastructure across domains but is domain-agnostic and does not address cost-aware conditional execution within a domain-specific pipeline. Our framework extends both by adding confidence-conditioned skip logic between skills, enabling early termination when upstream results are already sufficient.

D. Industrial AI and Failure Diagnosis

FailureSensorIQ [7] demonstrates that curated failure-mode knowledge bases achieve significantly higher diagnostic accuracy than parametric LLM reasoning alone, directly motivating our use of the FMSR agent's structured library as a first-pass confidence source before invoking TSFM inference. AssetOpsBench [1] itself defines the benchmark, the four MCP agent servers, the 141-scenario taxonomy, and the LLM-judge evaluation protocol we adopt. Its MetaAgent baseline, a plan-execute orchestrator using WatsonX Llama-4-Maverick, achieves approximately 48.8% average performance under controlled conditions; our work replaces its static execution logic with a skill-augmented, confidence-gated executor.

E. Confidence Estimation and Adaptive Computation

Early exit networks [8], mixture-of-experts routing, and cascade classifiers share the intuition that not all inputs require the same depth of processing, but these techniques have seen limited application to agentic pipelines where computation units are external API calls rather than neural network layers. Our confidence gate between FMSR and Deep-TSFM applies this principle directly to the agent setting: FMSR confidence gates TSFM invocation, with threshold θ controlling the accuracy-efficiency trade-off.

III. Methodology

A. Data

We evaluate on AssetOpsBench [1], an industrial O&M benchmark consisting of natural-language scenarios over Chillers, air-handling units (AHUs), Motors, Pumps, and Bearings. The benchmark combines four data modalities: sensor

time-series records, sensor metadata, failure-mode templates, and work-order or maintenance-policy information. The full benchmark contains 141 scenarios across four task families: fault diagnosis and action recommendation, anomaly detection, forecasting and preventive maintenance, and metadata retrieval. The final ablation run used a 54-scenario stratified input slice derived from the TSFM redundancy report. After AssetOpsBench judge filtering and complete rubric parsing, the condition-level metrics table reports 46 successfully graded scenarios per condition.

There is no train/validation/test split in the conventional supervised-learning sense. Our framework does not update model weights. Instead, it evaluates alternative inference-time orchestration policies over the same scenario bank. Threshold selection is reported as a sweep rather than as a learned validation procedure.

B. Model and Agent Architecture

The implemented system is a multi-agent orchestration stack rather than a single neural model [9]. It has three layers:

1) **LLM planner.** A WatsonX `llama-4-maverick-17b-128e-fp8` planner generates an ordered skill plan from the user query, with a heuristic fallback when an LLM provider is unavailable.
2) **Deterministic skill executor.** The executor runs skills from a registry, applies skip conditions, enforces cost budgets, stores intermediate context, and terminates early when a skill marks the task complete.
3) **MCP tool agents and knowledge plugins.** Tool calls are routed to the AssetOpsBench IoT, FMSR, TSFM, and WO agents. Knowledge plugins inject only the context required by the current skill: sensor metadata, failure modes, operating ranges, anomaly definitions, time-series metadata, and maintenance policies.

The system exposes seven executable skills: `data_retrieval`, `metadata_retrieval`, `anomaly_detection`, `root_cause_analysis`, `validate_failure`, `forecasting`, and `work_order_generation`. Each skill has a function, a cost estimate, a `should_skip` condition, and an optional `should_stop` signal. This structure makes trajectories deterministic, auditable, and comparable across ablation conditions. Figure 1 summarizes both the general skill-knowledge-agent stack and the fault-diagnosis path controlled by the FMSR confidence gate.

C. Optimization Scope: Training, Inference, or Both

The optimization scope is **inference-time orchestration only**. We do not train, fine-tune, quantize, prune, or distill the planner, evaluator, or TSFM models. The main optimization is deciding whether to invoke a costly external analysis component. In the fault-diagnosis pipeline, lightweight anomaly detection first flags abnormal sensor behavior. FMSR then maps the anomaly pattern to a candidate failure mode and the confidence evaluator computes a heuristic confidence score. If the score is at least θ, Deep-TSFM is skipped and the pipeline proceeds directly to validation and work-order generation. If the score is below θ, the executor invokes Deep-TSFM, re-runs FMSR on the refined anomaly picture, and then generates the work-order recommendation.

We evaluate $\theta \in \{0.50, 0.60, 0.65, 0.70, 0.80, 0.90, 0.95\}$ to map the accuracy-efficiency trade-off. Deep-TSFM runs only when FMSR confidence is strictly below θ; thus *smaller* θ skips more often (fewer Deep-TSFM calls), and *larger* θ skips only for the strongest FMSR scores and escalates more workloads to Deep-TSFM. Non-monotonicity in headline metrics can still arise from cost-budget skipping, incomplete sensor windows, judge variance, and the heuristic confidence signal.

D. Profiling Tools

Profiling is performed at the trajectory level. The main tools are:

- `trajectory_log.py`, which records per-step tool names, arguments, responses, latency, skill decisions, and summary metrics.
- `analyze_tsfm.py` (`src/get_trajectories/`), which audits baseline trajectories for redundant or wrong-domain TSFM invocations and generates `tsfm_report.csv`.
- `eval_runner.py`, which executes the ablation conditions and merges local run metrics with the AssetOpsBench evaluator output.
- `scripts/calibrate_costs.py`, which estimates median wall-clock skill costs on the active hardware and writes `skill_costs.json` for budget-aware execution.

The reported run, `colab_20260503_0230`, was executed on Google Colab GPU hardware (T4/A100 availability), with Python 3.12, MCP/stdio, LiteLLM, FastMCP, `uv`, WatsonX planner and judge endpoints, and IBM Granite TinyTimeMixer for Deep-TSFM inference.

E. Evaluation Metrics

We report both process metrics and final answer metrics. The AssetOpsBench/WatsonX judge uses `llama-3-3-70b-instruct` and grades six dimensions: task completion, data retrieval accuracy, generalized result verification, agent-sequence correctness, clarity and justification, and hallucination rate. A trajectory is marked *overall-correct* only when it passes all required rubric dimensions. We additionally track Deep-TSFM invocation rate, skills skipped, tool calls per task, total estimated skill cost, and end-to-end latency.

IV. Experimental Results

A. Experimental Setup

We compare 12 evaluated conditions: raw LLM prompting (A), a static tool baseline (B), planning-only execution without knowledge injection (C), skills plus knowledge without Deep-TSFM (D), skills plus knowledge with always-on Deep-TSFM (F), and seven full-system settings (E) with

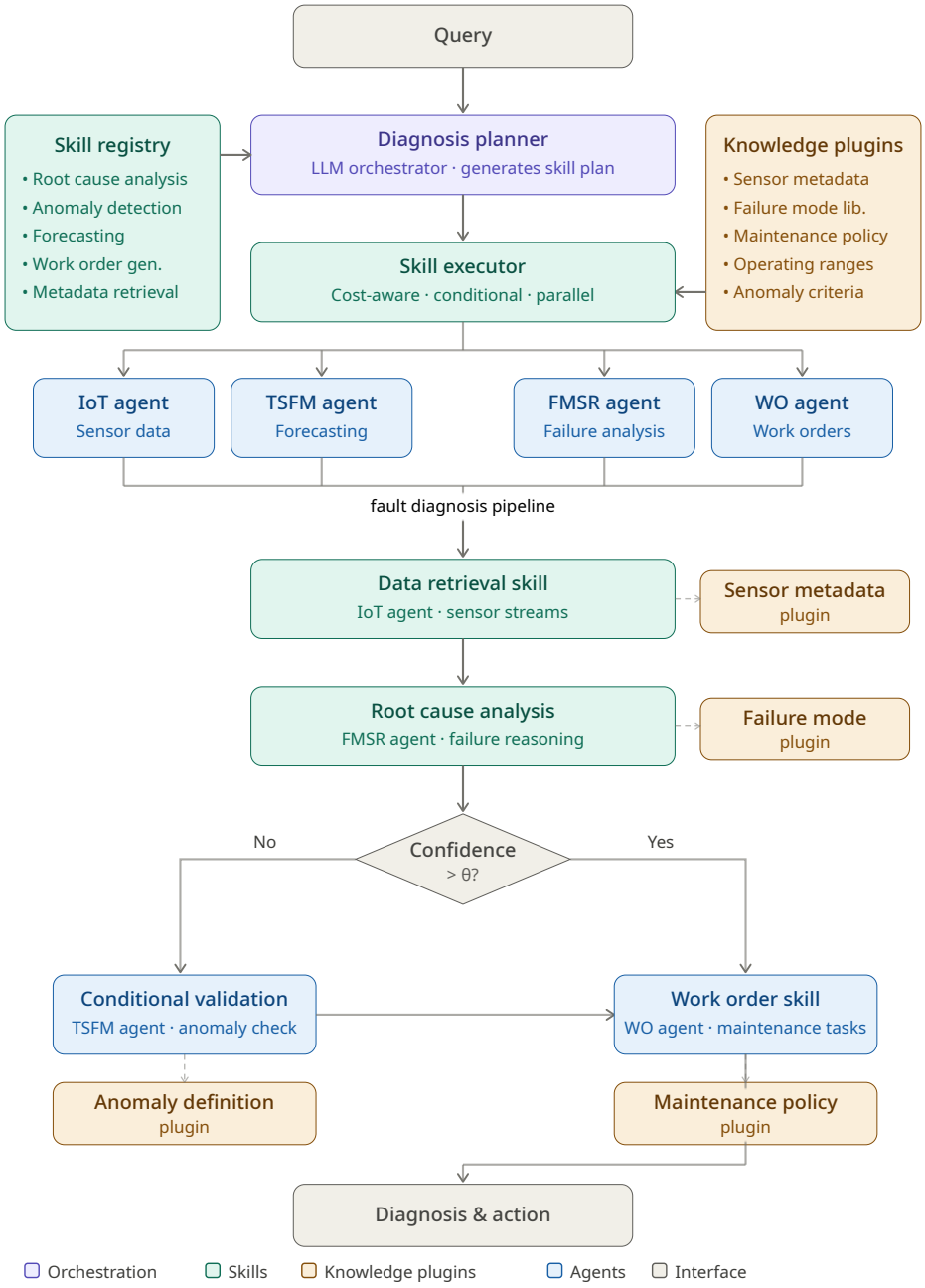

Fig. 1. Skill-knowledge-augmented agent architecture and fault-diagnosis execution path. The top half shows the planner, skill registry, skill executor, knowledge plugins, and four AssetOpsBench MCP agents. The lower half expands the fault-diagnosis workflow: data retrieval and FMSR root-cause analysis are executed first; high-confidence diagnoses proceed directly to work-order generation, while low-confidence cases invoke conditional TSFM validation before producing the final diagnosis and action.

$\theta \in \{0.50, 0.60, 0.65, 0.70, 0.80, 0.90, 0.95\}$. Each condition is evaluated with the same scenario inputs and the same judge rubric. The final condition summary contains 46 graded scenarios per condition, yielding 552 graded condition rows.

The ablation is designed to separate three effects: (1) whether structured skills improve process reliability over raw or static tool use, (2) whether scoped knowledge injection reduces hallucinations and improves grounding, and (3) whether adaptive Deep-TSFM gating outperforms both never-deep and always-deep execution.

B. *Before/After Comparison*

Table II summarizes the main before/after comparison. The best full-system setting is Condition E at $\theta = 0.8$, which obtains the highest overall-correct rate, 30.4%. This more than doubles the static tool baseline's 13.0% overall-correct rate and improves substantially over raw LLM prompting, which receives 0.0% overall-correct on the final scored run.

TABLE I
METHODOLOGY SUMMARY.

Component	Final setup
Data	AssetOpsBench industrial O&M scenarios over Chillers, AHUs, Motors, Pumps, and Bearings; 54-scenario input slice; 46 successfully graded scenarios in final metrics.
Model	WatsonX LLM planner + deterministic skill executor + IoT/FMSR/TSFM/WO MCP tool agents.
Optimization scope	Inference only; no model training or fine-tuning. Confidence gate controls Deep-TSFM invocation.
Profiling	Trajectory logger, TSFM redundancy auditor, ablation runner, cost calibration, WatsonX EvaluationAgent judge.
Metrics	Overall-correct, six rubric scores, hallucination rate, Deep-TSFM invocation, tool calls, skill cost, and latency.

The process improvements are larger than the final correctness improvements. Agent-sequence correctness rises from 6.5% in the raw LLM setting to 88.9% with the gated full system at $\theta = 0.8$, an 82.4 percentage-point gain. Hallucination rate decreases from 93.5% to 35.6%, a 57.9 percentage-point reduction. These results indicate that the main benefit of skills and plugins is not simply calling fewer tools; it is constraining the agent to a valid industrial workflow with relevant domain context.

C. Threshold Sweep

Table III reports the full-system threshold sweep. The best overall-correct operating point is $\theta = 0.8$. The neighboring thresholds show the expected trade-off is not perfectly monotonic because the threshold interacts with cost-budget skipping, missing data windows, judge variance, and the heuristic confidence proxy. Nevertheless, the sweep supports the key design choice: adaptive gating can outperform both no-deep and always-deep variants.

D. Analysis

The strongest result is that the gated full system at $\theta = 0.8$ is the best aggregate operating point. It improves overall-correct rate from 13.0% to 30.4% relative to the static-tool baseline, while maintaining a substantially lower hallucination rate. This supports the central hypothesis: in many industrial fault-diagnosis cases, FMSR's structured failure-mode knowledge is sufficient, and Deep-TSFM should be treated as an escalation path rather than a mandatory step.

The second result is that structured skill execution is the largest single source of process reliability. Condition D already improves agent-sequence correctness from 26.1% in the static baseline to 86.7% without invoking Deep-TSFM. The full system adds confidence gating and reaches 88.9% at $\theta = 0.8$. In other words, the skill layer fixes many misordered or incomplete trajectories before the TSFM gate is considered.

The third result is that always invoking Deep-TSFM is not optimal. Condition F obtains high task-completion and agent-sequence scores, but its overall-correct rate is only 17.4%, below both Condition D and the best gated Condition E. The likely explanation is that Deep-TSFM sometimes introduces partial or noisy anomaly outputs when the data window is insufficient; rerunning FMSR on that refined anomaly picture can degrade the final diagnosis. Adaptive gating avoids this pathway for high-confidence cases.

Finally, the trajectory audit confirms that TSFM misuse is primarily a routing problem. The baseline produced 54 unnecessary TSFM calls, 52 of which were wrong-domain calls. Thus, confidence gating alone cannot solve every inefficiency; the planner must also learn task-domain routing rules that avoid associating every anomaly-related keyword with time-series ML inference.

V. DISCUSSION

A. Interpretation of Results

The final evaluated results show that orchestration quality dominates raw model capability. Raw LLM prompting fails to satisfy all six rubric dimensions on any successfully graded scenario, while the static-tool baseline reaches 13.0% overall-correct. Adding deterministic skills and knowledge plugins without Deep-TSFM raises this to 21.7%, and the full gated system at $\theta = 0.8$ reaches 30.4%.

Skill structure provides the main reliability gain. The jump from Condition B to Condition D shows that replacing free-form or static tool usage with a skill registry produces much more valid execution traces. Agent-sequence correctness increases from 26.1% to 86.7% before Deep-TSFM gating is added. This supports the design decision to encode industrial workflows explicitly rather than relying on the LLM to rediscover the tool order at each step.

Knowledge plugins improve grounding. Knowledge plugins inject scoped sensor metadata, operating ranges, failure modes, anomaly definitions, and maintenance policies only when a skill requires them. This reduces noisy context and helps the planner and skill executor produce cleaner tool inputs. The hallucination rate falls from 73.9% in the static-tool baseline to 46.7% in the skills-plus-knowledge setting and to 35.6% in the best gated setting.

Adaptive Deep-TSFM beats always-deep execution. Always invoking Deep-TSFM is not the best use of the expensive model path. Although Condition F has strong process metrics, its overall-correct rate is 17.4%, below the gated $\theta = 0.8$ setting. This suggests that Deep-TSFM should be used as a selective escalation mechanism, especially when FMSR confidence is low or the lightweight anomaly picture is ambiguous.

The TSFM misuse pattern is a planning problem, not only a diagnostic problem. The redundancy audit found 54 unnecessary TSFM invocations, with 52 wrong-domain calls. This means that most misuse comes from tool-routing confusion: anomaly-related words trigger TSFM even when the task is actually work-order generation, metadata retrieval, or another non-TSFM category. The final framework addresses this with both skill-level routing and FMSR-confidence gating.

TABLE II
FINAL ASSETOPSBENCH JUDGE METRICS FOR REPRESENTATIVE ABLATION CONDITIONS. HIGHER IS BETTER EXCEPT HALLUCINATION RATE.

Condition	Overall	Task	Data	Verify	Seq.	Halluc.
A: Raw LLM	0.0%	6.5%	0.0%	6.5%	6.5%	93.5%
B: Static tool baseline	13.0%	19.6%	23.9%	17.4%	26.1%	73.9%
D: Skills + knowledge, no deep	21.7%	44.4%	33.3%	37.8%	86.7%	46.7%
E: Full system, $\theta = 0.8$	**30.4%**	**55.6%**	**46.7%**	**51.1%**	88.9%	35.6%
F: Skills + knowledge, always deep	17.4%	53.3%	44.4%	44.4%	**93.3%**	**33.3%**

TABLE III
FULL-SYSTEM THRESHOLD SWEEP FOR CONDITION E. HIGHER IS BETTER EXCEPT HALLUCINATION RATE.

θ	Overall	Task comp.	Agent seq.	Halluc.
0.50	26.1%	42.2%	86.7%	37.8%
0.60	15.2%	33.3%	85.7%	45.2%
0.65	28.3%	53.5%	83.7%	32.6%
0.70	10.9%	45.5%	88.6%	31.8%
0.80	**30.4%**	**55.6%**	88.9%	35.6%
0.90	26.1%	52.3%	**90.9%**	**31.8%**
0.95	28.3%	53.3%	80.0%	40.0%

B. Limitations

Several limitations constrain the conclusions that can be drawn from this work.

FMSR confidence is a heuristic proxy. The confidence score used for gating is computed from weighted signals rather than from a calibrated probability exposed by the FMSR server. It is useful for threshold sweeps, but θ should not be interpreted as a true posterior confidence.

Scored subset size. The final ablation was built from a 54-scenario stratified input slice, but the published metrics table reports 46 successfully graded scenarios per condition. Some scenario outputs were filtered or lacked complete parsed judge fields. Therefore, the strongest numerical claims should be read as applying to the final scored subset rather than to all 141 AssetOpsBench scenarios.

LLM judge variability. Scoring is performed by an LLM judge with a six-dimension rubric. Although the evaluator is held fixed across conditions, LLM judging can still be sensitive to prompt phrasing and output formatting. This may partly explain non-monotonic behavior in the θ sweep.

Data coverage and hardware dependence. The IoT path depends on exported CouchDB sensor data and per-asset CSVs. Deep-TSFM latency also depends heavily on GPU availability, and TSFM on CPU is impractical for large sweeps. Reported latency and cost trends should therefore be recalibrated whenever the deployment hardware changes.

C. Future Work

Learned confidence calibration. A lightweight calibrator trained on trajectory features could replace the current heuristic confidence proxy and choose asset-specific or failure-mode-specific thresholds.

Full benchmark scoring. The next evaluation should extend the complete LLM-judge cycle from the final scored subset to all 141 AssetOpsBench scenarios and report results by task category.

Additional confidence gates. The same adaptive execution principle can gate other boundaries, such as IoT sensor fetching, FMSR after anomaly detection, and WO generation after diagnosis validation.

Cross-asset generalization. The current implementation is strongest on chiller-style failures. Testing Pumps, Bearings, Motors, and AHUs separately would reveal whether the knowledge plugins transfer without per-asset retuning.

VI. CONCLUSION

We presented a skill-augmented and knowledge-aware agent framework for industrial fault diagnosis and action recommendation on AssetOpsBench. The project began from a concrete inefficiency: the baseline pipeline frequently invoked Deep-TSFM, the expensive statistical time-series inference component, even when the task domain or FMSR confidence did not justify it. A trajectory audit found 54 unnecessary TSFM invocations, and 96% of these were wrong-domain calls, showing that much of the inefficiency was caused by poor orchestration rather than by lack of diagnostic knowledge.

Our framework addresses this with deterministic skills, scoped knowledge plugins, and an FMSR-confidence gate before Deep-TSFM. The optimization is entirely inference-time: no model weights are trained or fine-tuned. Instead, the system improves reliability and efficiency by deciding which tool calls are actually needed for each scenario.

On the final evaluated run, the best gated condition (E, θ=0.8) reaches 30.4% overall-correct rate, compared with 13.0% for the static-tool baseline and 0.0% for raw LLM prompting. It also raises agent-sequence correctness from 6.5% to 88.9% and reduces hallucination rate from 93.5% to 35.6%. These gains support the central lesson of the project: in industrial agent systems, meaningful performance improvements can come not only from larger models, but from better orchestration. The strongest agent is not the one that runs every tool; it is the one that knows when a tool is unnecessary.

REFERENCES

[1] D. Patel, S. Lin, J. Rayfield, N. Zhou, R. Vaculin, N. Martinez, F. O'Donncha, and J. Kalagnanam, "AssetOpsBench: Benchmarking AI agents for task automation in industrial asset operations and maintenance," *arXiv preprint arXiv:2506.03828*, 2025.

[2] LangChain AI, "DeepAgents: A framework for building planning-based AI agents," GitHub repository, 2025. [Online]. Available: https://github.com/langchain-ai/deepagents

[3] X. Li, W. Chen, Y. Liu, S. Zheng, X. Chen, Y. He, Y. Li, B. You, H. Shen, J. Sun *et al.*, "SkillsBench: Benchmarking how well agent skills work across diverse tasks," *arXiv preprint arXiv:2602.12670*, 2026.

[4] A. Fourney, G. Bansal, H. Mozannar, C. Tan, E. Salinas, E. Zhu, F. Niedtner, G. Proebsting, G. Bassman, J. Gerrits *et al.*, "Magentic-One: A generalist multi-agent system for solving complex tasks," *arXiv preprint arXiv:2411.04468*, 2024.

[5] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.

[6] Y. Liang, R. Zhong, H. Xu, C. Jiang, Y. Zhong, R. Fang, J.-C. Gu, S. Deng, Y. Yao, and N. Zhang *et al.*, "SkillNet: Create, evaluate, and connect AI skills," *arXiv preprint arXiv:2603.04448*, 2026.

[7] C. Constantinides, D. Patel, R. Vaculin, N. Zhou, S. Lin, and J. Kalagnanam, "FailureSensorIQ: A multi-choice QA dataset for understanding sensor relationships and failure modes," in *Proc. Conf. Neural Information Processing Systems (NeurIPS)*, 2025.

[8] S. Teerapittayanon, B. McDanel, and H. T. Kung, "BranchyNet: Fast inference via early exiting from deep neural networks," in *Proc. 23rd Int. Conf. Pattern Recognition (ICPR)*, Dec. 2016, pp. 2464–2469.

[9] V. Mazeeva, M. Abbaszadeh, S. Arora, and S. Shejwal, "AssetOpsBench Final Project," GitHub repository, 2026. [Online]. Available: github.com/shreyarora2198/AssetOpsBench/tree/team14-final.

[10] V. Mazeeva, M. Abbaszadeh, S. Arora, and S. Shejwal, "SkillsAgent: Skill-augmented, knowledge-aware agents for AssetOpsBench," GitHub repository, 2026. [Online]. Available: github.com/verammaz/SkillsAgent.

APPENDIX A
AI USE DISCLOSURE

Per the HPML AI Use Policy, our team used AI assistance in completing this project. We used Claude, including Claude Code CLI and conversational Claude, and GitHub Copilot. Claude was used for repository-editing support, helper-script drafting, environment debugging, branch-integration planning, report prose refinement, and generating the final architecture figure from team-written prompts. GitHub Copilot was used for IDE-based code navigation and explanations while exploring the upstream AssetOpsBench repository.

AI assistance affected parts of the evaluation scripts, SkillsAgent tool wiring, Colab setup notebook, README files, dependency/debugging workflow, literature-review drafting, and the architecture figure. All substantive design decisions, experiments, reported metrics, interpretations, and final conclusions were made by the team.

We verified correctness by reading source files directly, running scripts end-to-end, rerunning environment setup and TSFM smoke tests, inspecting judge outputs, checking the final ablation artifacts, and reviewing the architecture figure against the implemented planner–skill executor–MCP-agent workflow. AI-suggested claims that were not supported by committed results were removed.

By submitting this report, the team confirms that the analysis, interpretations, and conclusions are our own, and that AI assistance has been disclosed.

[0]Project README for grading and reproduction: https://github.com/shreyarora2198/AssetOpsBench/blob/team14-final/HPML_README.md

The repository root contains the original upstream AssetOpsBench README from IBM; our course submission documentation is provided in `HPML_README.md`.

[1]We also published a companion Medium article describing the project motivation and system design: https://medium.com/@2003sans/rethinking-industrial-ai-agents-teaching-systems-when-to-stop-539ced0f614f

[2]AssetOpsBench Final Project repository: https://github.com/shreyarora2198/AssetOpsBench/tree/team14-final

[3]SkillsAgent repository: https://github.com/verammaz/SkillsAgent